

# **Maritime CargoService: Sprint 1**

Davide Chirichella, Gabriele Doti, Daniele  
Maccagnan

Ingegneria dei Sistemi Software, A.A. 2025/2026  
Alma Mater Studiorum . Università di Bologna

June 26, 2026

# 1. Introduction

Il punto di partenza di questo sprint è l'architettura logica e l'analisi dei requisiti definita nello Sprint 0 (recuperabile al seguente [link](#)<sup>o</sup>). Si riporta di seguito il goal dello sprint 1:

Realizzare un primo prototipo eseguibile del **cargoservice** che implementi il comportamento principale descritto dai requisiti mediante collaboratori simulati.

Al termine dello Sprint1 il sistema dovrà essere in grado di:

- ricevere una **load\_request** proveniente dall'IOPort;
- verificare lo stato della hold, dell'IOPort e del servizio;
- produrre una delle tre risposte previste:
  - **load\_accepted(slotID)**;
  - **load\_retrylater**;
  - **load\_refused**;
- mantenere il modello logico della hold e la prenotazione dello slot;
- gestire gli stati **engaged** e **disengaged**;
- pilotare il LED in funzione dello stato del sistema;
- superare i test funzionali definiti nello Sprint0.

In questa fase il cargorobot, il sonar, il markerdevice e l'IOPort saranno rappresentati tramite collaboratori simulati.

# 2. Requirements

L'azienda richiede di realizzare un servizio denominato **cargoservice** con il seguente funzionamento:

- Il cargoservice è in grado di ricevere una richiesta di carico di un container inviata da un cliente tramite il pulsante dell'IOPort.
- Invia la risposta **retrylater** se l'IOPort è attualmente occupato da un container oppure se il sistema è **Out of service**.
- Rifiuta la richiesta quando la hold è già piena, ovvero gli slot1-4 sono già tutti occupati.
- Altrimenti, considera il sistema come **engaged**, rileva uno slot libero e restituisce come risposta il nome dello slot riservato. Mentre il sistema è engaged, il LED deve lampeggiare.
- Quando la richiesta di carico viene accettata, il cliente deve spostare il container nell'area del sensore entro un tempo prefissato (ad esempio 30 secondi), altrimenti il sistema diventa **disengaged**.
- Successivamente, il cargoservice utilizza il cargorobot per spostare il container dall'IOPort a slot5 (per l'etichettatura del container) e poi allo slot riservato.
- Il servizio deve inoltre mostrare sul display dell'IOPort:
  - lo stato attuale della hold
  - il messaggio "**Service working**" quando tutto sta procedendo correttamente
  - il messaggio "**Out of service**" se il sensore sonar misura una distanza  $D > D_{FREE}$  per almeno 3 secondi (possibile guasto del sonar)

### 3. Requirement analysis

L'analisi dettagliata del dominio e dei requisiti è già stata affrontata nello Sprint 0, in questa fase ci concentriamo esclusivamente sul ciclo di gestione delle richieste di carico (**core business**) del sistema.

Come emerso in precedenza, il fulcro del sistema è l'attore **cargoservice**, la cui responsabilità principale è fungere da orchestratore per coordinare le operazioni.

Tuttavia, i requisiti affrontati in questo sprint presupporrebbero già l'implementazione e la presenza di altri componenti del sistema, come la stiva (hold), i sensori (sonar), i dispositivi di I/O (IOPort, LED, markerdevice) e l'interazione con l'hardware del robot (cargorobot). Per focalizzarci unicamente sulla logica di business e rispettare un processo di costruzione incrementale, in questo Sprint 1 verranno utilizzati dei componenti **simulati**

L'uso del linguaggio qak ci permetterà di modellare il **cargoservice** come un **attore autonomo**. Il sistema si avvarrà dei seguenti collaboratori simulati:

- **ioportmock**: simulerà il Customer. Avrà il compito di generare la `load_request` e di attendere le risposte formali del sistema (`load_accepted`, `load_refused`, `load_retrylater`).
- **sonarmock**: simulerà il rilevamento fisico del container, inviando al sistema un messaggio per notificare che l'area dell'IOPort è occupata. Simulerà inoltre eventuali eventi di guasto per forzare il sistema nello stato di Out of service.
- **ledmock**: simulerà il dispositivo fisico di segnalazione, limitandosi a ricevere e stampare a video i comandi operativi.
- **cargorobotmock**: fungerà da "simulatore" per il sistema di movimentazione, ricevendo la richiesta di movimento verso un target ed emettendo una fittizia risposta di completamento.

### 4. Problem analysis

Come emerso dai requisiti, questo componente funge da **orchestratore**: coordina le operazioni degli altri componenti del sistema al fine di eseguire le procedure di carico.

#### 4.1. Rappresentazione dello stato interno della stiva

In questo sprint la gestione della stiva non viene ancora delegata a un componente esterno reale. Il **cargoservice** mantiene quindi una propria rappresentazione locale dello stato degli slot.

La stiva viene modellata tramite un array interno all'attore:

```
var Slots = intArrayOf(0, 0, 0, 0)
```

Ogni posizione dell'array rappresenta uno slot della stiva. Il valore `0` indica uno slot libero, mentre un valore diverso da `0` indica uno slot occupato o riservato.

Questa scelta permette al **cargoservice** di verificare autonomamente se la stiva è piena, senza introdurre in questa fase un componente dedicato alla gestione della **hold**.

#### 4.1.1. Vocabolario delle Interazioni

Per quanto riguarda l'interazione tra **Customer** (simulato da `ioportmock`) e **CargoService** si utilizzano i messaggi già definiti in fase di Sprint 0:

```
Request load_request      : loadRequest(none)
Reply load_accepted       : loadAccepted(slotID) for load_request
Reply load_retrylater     : loadRetryLater(none) for load_request
Reply load_refused        : loadRefused(none) for load_request
```

Per quanto riguarda l'interazione con i Sensori simulati (Sonar e `IOPort`): Il **sonarmock** notificherà al sistema i cambiamenti fisici dell'ambiente (es. deposito del container o guasti). L'utilizzo di una `Dispatch` consente di modellarne la ricezione asincrona.

```
Dispatch set_ioport_status : setIOPortStatus(STATUS) // STATUS: "free" o
"occupied"
Dispatch set_service_status : setServiceStatus(STATUS) // STATUS: "working"
o "outofservice"
```

Interazione con gli Attuatori simulati (LED e Robot): Il **cargoservice** comanda il lampeggio del LED e attende in modo sincrono o asincrono il termine dei movimenti del `cargorobotmock`.

```
Dispatch led_ctrl : ledCmd(CMD) // CMD: "on", "off", "blink"

Request robot_move : robotMove(TARGET) // TARGET: "slot5", "slot1", ecc.
Reply robot_done : robotDone(none) for robot_move
```

L'attore **cargoservice** è, già da Sprint 0, inteso come una Macchina a Stati Finiti che utilizza variabili interne per mantenere la conoscenza dello stato applicativo:

```
[#
  var IOPortOccupied = false
  var ServiceWorking = true
  var CargoState      = "disengaged"
  var ReservedSlotId  = -1
  var Slots           = intArrayOf(0, 0, 0, 0)
#]
```

Si gestisce quindi il ciclo di vita della richiesta seguendo Le transizioni principali:

```
QActor cargoservice context ctxcargoservice {
  ... // Inizializzazione variabili
```

```

    State disengaged {
        println("cargoservice | DISENGAGED: waiting for load_request...") color
blue
    }
    Transition t0
        whenRequest load_request → handle_load_request
        whenMsg set_ioport_status → update_ioport_status

    State handle_load_request {
        // Controllo stato servizio
        if [# !ServiceWorking || IOPortOccupied #] {
            replyTo load_request with load_retrylater : loadRetryLater(none)
        } else {
            // Controllo stiva
            [#
                var slotFound = false
                for (i in 0..3) {
                    if (Slots[i] == 0) {
                        ReservedSlotId = i + 1
                        Slots[i] = 1 // Prenota logicamente
                        slotFound = true
                        break
                    }
                }
            #]

            if [# slotFound #] {
                replyTo load_request with load_accepted :
loadAccepted($ReservedSlotId)
                forward ledmock -m led_ctrl : ledCmd(blink)
                [# CargoState = "engaged" #]
            } else {
                replyTo load_request with load_refused : loadRefused(none)
            }
        }
    }
    Goto engaged if [# CargoState == "engaged" #] else disengaged

    State engaged {
        println("cargoservice | ENGAGED: waiting for container to be placed...")
color magenta
    }
    Transition t0
        whenMsg set_ioport_status → start_robot_operation

    ...
}

```

- 5. Test plans
- 6. Project
- 7. Testing
- 8. Deployment
- 9. Maintenance

## 10. Team di lavoro

| NOME E COGNOME     | MATRICOLA  |
|--------------------|------------|
| Davide Chirichella | 0001222371 |
| Gabriele Doti      | 0001245897 |
| Daniele Maccagnan  | 0001247340 |